FraudScope review and mastery guide

FraudScope: Fraud Detection with Temporal Graph Neural Networks

Sileshi Hirpa · October 2026

1. The explanation to master first

FraudScope studies how a team with limited analyst capacity can rank suspicious transactions, justify reviews and preserve reproducible evidence. It combines a synthetic transaction simulator, chronological model evaluation, temporal graph experiments and a working private investigation application. The active model is calibrated logistic regression. A risk-gated temporal GNN produces shadow scores. Nothing automatically blocks a payment.

The measured result is deliberately qualified: across three synthetic populations of 60,000 events, relational aggregation had mean fraud recall of 53.95% within the top-scored 5%, compared with 50.69% for gradient boosting. No GNN won overall validation cost. Real IEEE-CIS performance remains unmeasured. Be able to explain that tradeoff without treating the graph model as the automatic winner.

Before an interview, reproduce the application checks, trace one transaction through source, explain one leakage test, and rehearse the private demo. This AI-assisted reference becomes useful portfolio evidence when you can defend your own understanding and changes.

2. Repository map and two execution paths

The root Python package is the research and FastAPI reference. Read `fraudscope/synthetic.py`, `features.py`, `splits.py`, `pipeline.py`, `models.py`, `metrics.py` and `service.py` in that order. The neural path adds `graph_data.py`, `graph_teacher.py`, `gnn.py` and `graph_experiment.py`.

`applications/workbench` is the deployed React/Workers application. `app/workbench.tsx` renders the analyst interface. `lib/api.mjs` validates events, enforces ownership and commits state. `lib/inference.mjs` derives features and runs exported logistic/neural weights. `model-data` contains weights, historical evidence and independent parity fixtures. `drizzle` contains the database schema migration. Tests use a SQLite adapter for the same handler.

These paths share modeling ideas but are different services. The Python model is a trusted joblib bundle generated by research training. The workbench consumes JSON-exported parameters. Its shadow GNN can run without a Python process or GPU. The browser does not train models.

3. Data and point-in-time reasoning

Synthetic events contain transaction ID, timestamp, account and device identifiers, amount, reported account/email age, geographic distance, login failures, channel, product and device type. The simulator includes recurring entities, shared devices, suspicious scenarios, drift and delayed labels. These choices permit controlled tests; they cannot establish the prevalence, behavior or costs of a real merchant.

An event at time t can use only records with time strictly less than t. Equal-time events are excluded from history as well as future ones. Examples include account counts in the prior hour/day, amount relative to the prior seven-day mean, historical device-account sharing and a new-device flag. Missing entity identifiers must not create a giant shared node.

A useful exercise is to alter every future event and every fraud label while keeping the current event and prior history fixed. Earlier feature vectors and graph neighborhoods should not change. Read the corresponding feature and graph tests. Explain why computing account counts over the entire dataset before splitting would leak future behavior.

4. Chronological model evaluation

Training fits preprocessing and model parameters. Calibration maps scores to probability estimates on a later interval. Validation selects the candidate and threshold. Final testing evaluates those frozen choices. The split is based on elapsed time, not fixed row counts, so exact sample proportions differ from 55/15/10/20%. Purges account for a two-day assumed label delay.

Do not fit imputers, scalers, categorical encoders or the probability calibrator on the test interval. Do not repeatedly choose settings by looking at test results. A separate calibration interval reduces reuse of labels for fitting both the predictor and the decision policy, although limited calibration data can still yield noisy estimates.

The causal risk teacher needs its own timing discipline. Historical supervised risk signals cannot be generated by a model trained on labels that were not yet available at the historical score time. The implementation uses expanding past-only teacher blocks and matured labels. The first block uses a fixed prior when no fitted teacher is available. Later scoring uses the past-trained teacher, not target labels.

The two-day lag is a sensitivity assumption. It is not a documented property of IEEE-CIS and does not prove that every anonymized benchmark feature was available in real time.

5. Models and the correct controls

Logistic regression provides a transparent tabular baseline with learned linear effects after preprocessing. Histogram gradient boosting captures nonlinear tabular interactions. Isolation Forest offers an anomaly-based comparison; anomaly and fraud are not synonyms.

The no-edge MLP has neural nonlinearities but receives no graph messages. It still sees engineered historical behavioral features. It isolates the added value of learned neighbor aggregation, not the value of

all historical information.

Graph nodes represent transactions. Historical relations connect earlier transactions sharing an account within seven days or a device within one day. The model keeps the latest eight neighbors per relation. Relational GraphSAGE-style aggregation learns from typed mean messages. Temporal relation attention lets the target weight historical messages with relative-time information. Risk-gated attention also modulates messages using past-trained teacher risk.

Read the implementation before deriving equations. In the workbench, risk gates are bounded positive values based on a learned scale, threshold and historical risk. Attention is relation-aware and uses time. These are custom one-hop models. Do not describe them as a full Heterogeneous Graph Transformer or a memory-based Temporal Graph Network. Those papers motivate directions; they are not reproduced architectures here.

A shared device can reflect household use, a shared terminal or fraud coordination. A graph edge is evidence of a relationship proxy, not proof that two people are the same entity or have committed fraud.

6. Metrics and operating decisions

Average precision summarizes precision at increasing recall, with scikit-learn's step-weighted definition. It is not trapezoidal area under a precision-recall curve. ROC-AUC measures pairwise ranking and can appear strong when rare-event review quality remains inadequate. Brier score evaluates probability errors but also depends on discrimination and outcome uncertainty; it is not purely calibration error.

Recall within the top 5% sorts holdout scores and inspects a fixed number of transactions. It answers a capacity-constrained ranking question. A validation-selected fixed threshold answers a different deployment question. Under drift, that threshold can review more or less than 5% of future transactions. It cannot guarantee capacity. A strict top-k queue would need its own operational design, latency assumptions and evaluation.

Estimated cost combines modeled missed-fraud loss and review costs. It is not observed business savings. A model can improve top-5% recall but lose on validation cost because score calibration, amount concentration and threshold choices differ. This explains why the graph candidates did not win the overall selection rule.

The three seeds are independent simulator populations with the same experimental configuration. Sample standard deviation describes variation across those populations. It is not a confidence interval or proof that a small model difference is significant. The relational model's 1.11 percentage-point advantage over the no-edge MLP is a descriptive result worth studying, not a general superiority claim.

7. Structural stability experiment

Freeze model weights, the probability calibrator and the decision threshold. Delete a fraction of eligible historical links or rewire them to strictly earlier sources. Then measure several different quantities: changed adjacency, embedding displacement, absolute probability change and the fraction crossing the fixed review threshold.

Graph displacement does not equal prediction displacement. A large representation change can leave a saturated probability almost unchanged. A small score change near a threshold can flip a review decision. Inspect both continuous drift and discrete decisions rather than using one robustness number.

Rewiring preserves a past-only direction but can violate entity membership and original relation windows. It is a controlled integrity stress, not a realistic adversarial threat model. Frozen-weight sensitivity is different from retraining stability. Neither establishes a certified robustness bound or a theorem about GNN convergence.

The recovered workbench includes 12 seed-42 raw stress draws. The presentation's three-seed means were separately recovered from its editable chart. The raw multi-seed files have not been recovered, so that limitation is recorded with the chart. A new full research run can regenerate raw outputs, but its results must be recorded as a new run.

8. Trace a transaction through the workbench

Start with input validation. Unknown fields, invalid IDs, non-finite numbers and out-of-range values are rejected. A trusted authenticated-user header identifies the workspace in the hosted flow. Local tests inject that identity directly and therefore do not validate the identity gateway.

The API loads strictly earlier matching history, derives features, encodes them, computes the logistic probability and active review decision, computes the historical teacher risk and GNN shadow score, and creates a deterministic evidence brief. The explanation is a rule-linked evidence summary, not a generative AI finding or a proof of causality.

The transaction, derived evidence and audit record commit atomically with workspace revision checks. Exact retries return the stored response. Reusing an ID with a different payload fails. Late events are rejected rather than silently revising history. An analyst review includes an expected revision to prevent lost updates. A stale review must refresh before saving again.

Ownership scopes reads and writes. CSV exports escape formula-like text. Audit verification checks the internal hash chain and materialized data consistency. It detects the tested edits and chain truncation, but a fully privileged actor could rewrite all records and the head without an external anchor.

9. Reproduce and verify

Use `docs/HIRING_MANAGER_REVIEW.md` for exact first-clone and existing-checkout commands. Public cloning needs no GitHub login. Preserve local changes before any fast-forward pull. Record the branch and commit for every demonstration.

For the workbench, install pnpm 11.25.0 under Node 22.13+, run frozen installation, copy the hosting example without overwriting an existing config, then run tests, TypeScript and the local reviewer tour. Expect 12 passing application tests and REVIEWER TOUR PASSED with 24 events and 25 audit records. Independent parity covers 80 fixture cases and a 48-event raw-event replay, including prior features and GNN inference.

For Python, use a separate Python 3.12 environment with the development lockfile and editable installation. The fresh Intel Mac recovery run passed 41 tests with one PyTorch module skipped. Use the declared Linux CPU lock for GNN testing and training. The historical 50-check total is not a fresh Mac result.

Run the tabular demo before expecting a Python service model or checkpoint-dependent exports. For matching research scope, use 60,000 rows, seeds 42/73/101 and 12 neural epochs. A 12,000-row, three-epoch run is a smoke experiment with different metrics. Keep it in a separate output directory. Never overwrite historical outputs merely to obtain a desired score.

The notebooks contain historical execution outputs. Their re-execution was not completed during this recovery. Review missing-artifact assumptions before running them. The original full archive, checkpoints and some reports remain pending recovery.

10. Demonstration script

Open with the business constraint and synthetic-data boundary. Show the active policy and shadow model distinction. Replay the bounded feed, open a transaction and explain two concrete features. Inspect its historical graph, compare neural scores and save an analyst review. Export records and verify the audit. Finish with the cost-selection result and the real-data evaluation gap.

The hosted demo is owner-private. Arrange a screen share rather than promising an unrestricted link. Repeated replay clicks add events to the current workspace. For a reproducible empty-state backend demonstration, use the local reviewer tour, which operates on isolated SQLite storage.

Rehearse a fallback using the PowerPoint, PDF and terminal tour. State clearly when you are showing saved results versus live behavior. Do not characterize in-process API checks as load testing or full hosted browser testing.

11. Interview questions to practice

Why use a graph? Shared account/device history can expose relational patterns. The measured incremental contribution must be tested against both strong tabular baselines and a no-edge neural control.

Why retain logistic regression in the app? The live policy is the original calibrated operating reference. The later graph comparison did not justify automatic promotion. Promotion requires an explicit selection policy and separate deployment validation.

How did you prevent leakage? Cite strictly earlier feature/neighbor logic, training-only preprocessing, chronological stages, delayed-label purges and a causal historical risk teacher. Then acknowledge uncertain feature timing in real anonymized data.

Why not use labels as edge weights? A target label is unavailable at scoring time and would leak outcomes. Historical teacher predictions use only information legitimately available then.

Is the GNN robust? It has controlled frozen-weight structural stress evidence. Those tests reveal sensitivity under specified edits, not universal robustness, certified bounds or realistic attacker success rates.

Does the application explanation prove fraud? No. It summarizes observed signals and policy context. The probability, analyst assessment and confirmed mature outcome are different objects.

How would you deploy on real data? Establish authorization and lineage, validate feature availability, reproduce temporal evaluation, tune cost/capacity with accountable owners, assess subgroups and unseen entities, and validate operations before any financial decision use.

What did AI assistance contribute? Describe it candidly. Explain the source you reviewed, checks you ran, decisions you changed and limitations you preserved. Claim personal ownership only for work you can demonstrate.

12. Learning exercises and next experiments

First, hand-compute one cold-start feature vector and a second event's historical counts. Confirm it against `extractFeatures`. Next, alter future timestamps and demonstrate that earlier features and graph messages remain unchanged. Then compare MLP and graph predictions after deleting all links, keeping weights fixed.

Reproduce the 5% ranking metric from saved scores and compare it with the fixed-threshold review rate. Explain why they differ. Change the cost assumptions in a separate validation experiment and record how the chosen threshold changes without reusing the final test for tuning.

For stability research, distinguish adjacency displacement from embedding and decision drift, inspect cases close to the threshold, and design a realistic link-error process. A later matched-retraining study should separate optimization variability, data variability and structural perturbation. Write the hypothesis and protocol before looking at the new test results.

For authorized IEEE-CIS data, inspect `TransactionID` joins, missing identity matches and proxy semantics. Avoid treating an artificial schema fixture as benchmark evidence. No real benchmark score should appear in the portfolio until a reproducible authorized run exists.

Sources and evidence

Implementation and numerical sources are in the repository, especially `applications/workbench/model-data/evidence.json`, `parity-report.json`, `docs/RECOVERY_STATUS.md` and `reports/recovery/structural_chart.json`. Research attribution is documented in `docs/SOURCES.md`.

Primary sources verified during recovery include RGH-GNN (https://arxiv.org/abs/2610.27107), BRIGHT (https://arxiv.org/abs/2205.13084), TGN (https://arxiv.org/abs/2006.10637), HGT (https://arxiv.org/abs/2003.01332), and THGT-FD (https://arxiv.org/abs/2609.07100). They motivate design discussion; their dataset scores are not FraudScope results. IEEE-CIS's official data source is https://www.kaggle.com/competitions/ieee-fraud-detection/data.